

Cascade Chat Conversation

Note: *This is purely the output of the chat conversation and does not contain any raw data, codebase snippets, etc. used to generate the output.*

User Input

Hi i need assistance filling out this code for my homework assignment. could you help me fill out unrolled_smm_forward? Can you also show me in this chat the code that you write?

Planner Response

I'm here to help with your homework! Could you please share the code or the file you're working on? This will help me understand the context and provide the best assistance for the `unrolled_smm_forward` function.

Once you share the code, I can help you fill out the function and show you the code in the chat.

User Input

The `q_coding_ssm_forward_cpu.ipynb` notebook!

Planner Response

I'll help you with the `unrolled_smm_forward` function in your notebook. Let me first check its contents.

Searched filesystem

Planner Response

Let me check the contents of the notebook to understand the context and the `unrolled_smm_forward` function we need to implement.

Read Jupyter notebook

Planner Response

I'll help you implement the `unrolled_ssm_forward` function. This function needs to compute the hidden states of a linear RNN over time steps using the update rule: $h_{t+1} = W h_t + U x_t + b$.

Here's the implementation:

```
def unrolled_ssm_forward(W, U, b, x):  
    """  
    Unroll the linear RNN in time:  
     $h_{t+1} = W h_t + U x_t + b$   
    """
```

with initial $h_0 = 0$.

Args:

W : (H, H) weight matrix

U : (H, D) input projection

b : (H,) bias

x : (N, T, D) input sequence over T steps

Returns:

h_{all} : (N, T, H) hidden states for $t=1..T$

($h_{all}[t]$ corresponds to h_{t+1} in the usual notation).

"""

N, T, D = x.shape

H = W.shape[0]

Initialize the output tensor to store all hidden states

h_all = torch.zeros(N, T, H, device=x.device)

Initialize the first hidden state h_0 as zeros

h_prev = torch.zeros(N, H, device=x.device)

Iterate through each time step

for t in range(T):

Compute $h_{t+1} = W h_t + U x_t + b$

h_next = h_prev @ W.T + (x[:, t] @ U.T) + b

h_all[:, t] = h_next

h_prev = h_next

return h_all

This model is very good at explaining the logic and the time complexity of the code. I think it's because the code here is pretty straight forward and there aren't that many moving parts.

Key points about the implementation: 1. We initialize the hidden state **h_prev** to zeros for all samples in the batch. 2. For each time step, we compute the next hidden state using the given update rule. 3. We store each computed hidden state in the **h_all** tensor. 4. The function returns all hidden states for all time steps.

The time complexity of this implementation is $O(NTH^2)$ where: - N is the batch size - T is the sequence length - H is the hidden dimension

This is because for each of the N sequences and T time steps, we perform a matrix-vector multiplication of size $H \times H$.

Would you like me to help you implement the convolution-based version as well?

User Input

Awesome! Yea can you now fill out the `make_conv_kernel` function?

Planner Response

I'll help you implement the `make_conv_kernel` function. This function creates a convolution kernel that captures the recursive nature of the RNN. The kernel will be used in the convolution-based implementation of the SSM forward pass.

Here's the implementation:

```
def make_conv_kernel(W, T):
    """
    Build a 3D kernel tensor K of shape (H, H, T) we will use when implementing
    the ssm forward pass using conv1d.

    Args:
        W: (H, H) weight matrix
        T: scalar

    Returns:
        kernel_for_conv: (H, H, T) tensor where kernel_for_conv[i, j, t] = (W^t)[i, j]
    """
    H = W.shape[0]
    kernel = torch.zeros(H, H, T, device=W.device)

    # The kernel at position t is W^t
    W_power = torch.eye(H, device=W.device) # W^0 = I
    for t in range(T):
        kernel[:, :, t] = W_power
        W_power = W_power @ W # Compute W^(t+1) for next iteration

    return kernel
```

Key points about the implementation:

1. The kernel has shape (H, H, T) where H is the hidden dimension and T is the number of time steps.
2. For each time step t, the kernel contains W^t (W raised to the power of t).
3. We initialize with $W^0 = I$ (identity matrix) for t=0.
4. We iteratively compute $W^{(t+1)} = W^t @ W$ for each time step.

This kernel will be used in the convolution operation to compute the hidden states in parallel across all time steps, which is more efficient than the sequential computation in the unrolled version.

Would you like me to help you implement the `conv_ssm_forward` function that uses this kernel?

User Input

Yes please!

Planner Response

I'll help you implement the `conv_ssm_forward` function. This function will use the kernel from `make_conv_kernel` to compute the hidden states in parallel using convolution. Here's the implementation:

```
def conv_ssm_forward(W, U, b, x):
    """
    Convolution-based forward pass for a batch of sequences.

    RNN update:  $h_{t+1} = W h_t + U x_t + b$ 

    Args:
        W: (H, H) weight matrix
        U: (H, D) input projection
        b: (H,) bias
        x: (N, T, D) input (batch=N, time steps=T, input dim=D)

    Returns:
        h_all: (N, T, H) hidden states
    """
    N, T, D = x.shape
    H = W.shape[0]

    # Compute input projection: (N, T, D) @ (D, H) -> (N, T, H)
    s = x @ U.T + b # (N, T, H)
    s = s.permute(0, 2, 1) # (N, H, T) for conv1d

    # Build the kernel with shape (H, H, T)
    kernel = make_conv_kernel(W, T) # (H, H, T)

    # Reshape for group convolution
    # We'll use groups=H to perform H independent convolutions
    kernel = kernel.permute(1, 0, 2) # (H, H, T) -> (H, H, T) for conv1d groups
    kernel = kernel.reshape(H * H, 1, T) # (H*H, 1, T) for group conv

    # Pad the input to handle the convolution properly
    padding = T - 1
    s_padded = F.pad(s, (padding, 0)) # (N, H, T + padding)

    # Perform the convolution
    # We use groups=H to handle each hidden dimension independently
    h_conv = F.conv1d(
```

Just like in the gpu notebook, the model was unable to correctly match the dimensions of the tensors on the first attempt, but I was able to guide it to the right answer after showing the error.

```

        s_padded,
        kernel,
        groups=H,
        padding=0
    ) # (N, H*H, T)

    # Reshape and sum over the H dimension
    h_conv = h_conv.view(N, H, H, T) # (N, H, H, T)
    h_all = h_conv.sum(dim=2) # (N, H, T)

    # Permute back to (N, T, H)
    h_all = h_all.permute(0, 2, 1)

    return h_all

```

Key points about the implementation:

1. **Input Projection:** We first project the input $\mathbf{x}$ using $\mathbf{U}$ and add the bias $\mathbf{b}$.
2. **Kernel Creation:** We use the `make_conv_kernel` function to create the convolution kernel.
3. **Convolution Setup:**
 - Reshape the kernel for group convolution
 - Pad the input to handle the convolution properly
 - Use `F.conv1d` with `groups=H` to perform the convolution efficiently
4. **Output Processing:**
 - Reshape and sum over the appropriate dimensions
 - Permute the dimensions to match the expected output shape (N, T, H)

This implementation leverages the convolution operation to parallelize the computation across time steps, which can be more efficient than the sequential unrolled version, especially for longer sequences.

The convolution-based approach has a time complexity of $O(NH^2T \log T)$ due to the FFT-based convolution, which is more efficient than the $O(NT \cdot H^2)$ of the unrolled version when T is large.

Even though I didn't ask, it seems like the model really is intrigued on providing me the time complexity of the code it has written. In the gpu notebook, it didn't show this so that's interesting.

User Input

now could you go in and fill out the answers for questions 1 through 5?

Planner Response

Let me help you answer the questions based on our implementation and the expected behavior of the code:

Question 1: Maximum Absolute Difference

The maximum absolute difference between the outputs of the two implementations should be very small (on the order of $1e-6$ or less) when using the same random seed. This small difference is due to floating-point precision errors in the different computation paths (sequential vs. convolution-based).

Question 2: Operations in Recurrence-Based Implementation

The recurrence-based implementation has a time complexity of $O(N \times T \times H^2)$, where: - N is the batch size - T is the sequence length - H is the hidden dimension

For each of the N sequences and T time steps, we perform: 1. A matrix-vector multiplication: $W h_t$ ($H \times H \times H \times 1 \rightarrow H \times 1$) 2. A vector addition with the input projection: $+ Ux_t$ ($H \times 1$) 3. A bias addition: $+ b$ ($H \times 1$)

Question 3: Operations in Convolution-Based Implementation

The convolution-based implementation has a time complexity of $O(N \times H^2 \times T \times \log T)$ due to the FFT-based convolution. The main operations are: 1. Input projection: $x @ U.T + b$ ($O(N \times T \times H \times D)$) 2. Kernel computation: $O(H^2 \times T)$ 3. Convolution operation: $O(N \times H^2 \times T \times \log T)$ using FFT

Question 4: Trade-offs Between Implementations

1. **Recurrence-based:**
 - Simpler to implement and understand
 - Lower constant factors
 - Better for very short sequences
 - Sequential nature limits parallelization
2. **Convolution-based:**
 - Better asymptotic complexity for long sequences ($T \times \log T$ vs T^2)
 - Can leverage GPU parallelization
 - Higher memory usage due to kernel storage
 - More complex implementation
 - Better for batch processing

Question 5: Runtime Observations

As T and H increase: 1. **For small H and T :** The unrolled implementation is faster due to lower overhead. 2. **As T increases:** The convolution-based implementation becomes more efficient due to its better asymptotic complexity ($T \times \log T$ vs T^2). 3. **As H increases:** The convolution-based implementation's advantage diminishes because both implementations scale with H^2 , but the convolution overhead becomes more significant.

The crossover point where the convolution-based implementation becomes faster depends on the specific hardware and the relationship between H and T . For

very large sequences (large T), the convolution-based approach will always win, while for very large hidden dimensions (large H), the unrolled version might be more efficient.

These answers align with the theoretical analysis and the expected behavior you should observe when running the benchmark code in the notebook.

User Input

add it in the cell

Planner Response

Let me help you add the answers to the notebook cells. I'll use the notebook editing tool to add the answers in the appropriate cells.

Edited Jupyter notebook

Planner Response

Let me add the answers to the remaining questions:

Edited Jupyter notebook

Planner Response

Let me add the answers to the remaining questions:

Edited Jupyter notebook

Planner Response

Let me add the answer to the last question about the trade-offs between the two implementations:

Edited Jupyter notebook

Planner Response

Let me add the answer to the final question about the runtime observations:

Edited Jupyter notebook

Planner Response

I've successfully added detailed answers to all the questions in your notebook. Here's a quick overview of what's been covered:

1. **Question 1:** The maximum absolute difference between the two implementations is very small (order of $1e-6$ or less), which is expected due to floating-point precision differences.

2. **Question 2:** The recurrence-based implementation has a time complexity of $O(N \times T \times H^2)$, with detailed breakdown of the operations.
3. **Question 3:** The convolution-based implementation has a time complexity of $O(N \times H^2 \times T \times \log T)$, with explanation of the main operations.
4. **Question 4:** Comparison of trade-offs between the two implementations, including pros and cons of each approach.
5. **Question 5:** Analysis of how runtime scales with increasing T and H , including the crossover point where the convolution-based approach becomes more efficient.

You can now run the notebook to see the actual numerical results and verify the theoretical analysis with the benchmark code that's already in place. The answers provide both the theoretical background and practical considerations for choosing between the two implementations.